

Autonomous Drone Racing

Session 3: Algorithms

06.05.2026

Marcel Rath

*Chair of Safety, Performance and Reliability for Learning Systems
School of Computation, Information and Technology*



Competition Results



- Incredible results this term!
- We've admitted the first 12 teams
- You may continue to submit to the leaderboard (does not count towards your final evaluation)

Rank	Team	Time [s]	Success Rate [%]	Submissions
1	Nikola & Moritz	4.896	55	9
2	Millennium Falcon	5.404	50	7
3	Wei Lun & Lars	5.520	55	6
4	Theo & Olcay	5.606	50	9
5	Timo & Jeffrey	5.829	55	21
6	Ka-Fa 1500	5.864	50	5
7	Xiaochen & Aoji	6.246	50	4
8	Leonie & Nicolas	6.291	65	9
9	XYZ	6.486	50	3
10	Deniz & Ahmet	6.494	50	10
11	Thunderstrike	6.774	70	2
12	Sandro & Marek	7.072	50	3



Final Challenge

- The goal at the end of the term is to beat **level 3** (track layout is given at runtime)
- The final evaluation will include
 - Success rates in simulation (N runs with a fixed seed)
 - Best lap time in real (we will provide the final track)
 - More details later in the term



What We Cover Today

1. Choosing The Right Method [10 min]
2. Path Planners [15 min]
3. Model Predictive Control [15 min]
4. Sampling-Based Model Predictive Control [10 min]
5. Deep Reinforcement Learning [15 min]
6. Next Steps & Questions [5 min]



Choosing The Right Method

- Factors that should impact your decision:
 - Group skillset (e.g., optimization → planning, control → MPC)
 - Resources (e.g., compute, time)
 - Reference implementations
 - Paper materials
- Factors that should **not** impact your decision:
 - This method sounds cool
 - Others are also using this approach



Manage Your Risks

If you...

- Implement something for the first time
- Do not find reference implementations online
- Need to use advanced language capabilities (e.g. C++/Rust bindings) and have never done so
- Are new to Python

... you are taking a lot of risks

Manage Your Risks



Instead:

- Ideally choose a method that you know
- Build something that works within 1-2 weeks
 - Start with a minimum viable implementation
 - Doesn't need to run on the race yet
- Have a good understanding of how to improve things incrementally
 - Extensions without throwing away your progress
 - Example: Build an MPC with CasADi, then switch to acados



Rough Timeline

- Challenge deadline: Have something that *just works* on level 2
- First lab session: Your method is (nearly) running in sim (level 3)
- Progress presentation: The basic method is working in sim
- Next lab sessions: Refine and make it work on real hardware

If you fall significantly behind this schedule, simplify your approach!



What We Cover Today

1. Choosing The Right Method [10 min]
2. Path Planners [15 min]
3. Model Predictive Control [15 min]
4. Sampling-Based Model Predictive Control [10 min]
5. Deep Reinforcement Learning [15 min]
6. Next Steps & Questions [5 min]



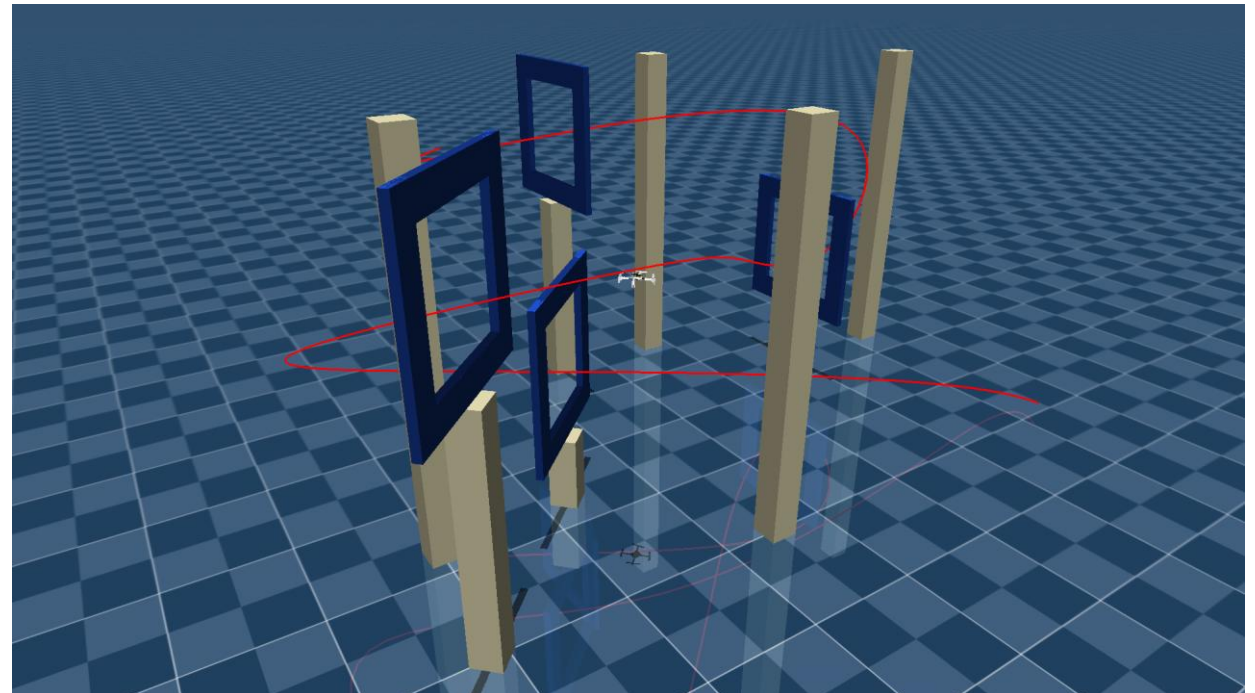
What We Cover Today

1. Choosing The Right Method [10 min]
2. Path Planners [15 min]
3. Model Predictive Control [15 min]
4. Sampling-Based Model Predictive Control [10 min]
5. Deep Reinforcement Learning [15 min]
6. Next Steps & Questions [5 min]



Path Planners

- Objective: Plan a fast and feasible trajectory through the track
- Best interface: state
- Pros:
 - Simplicity of implementation
 - Interpretability
- Cons:
 - Speed
 - Less direct control





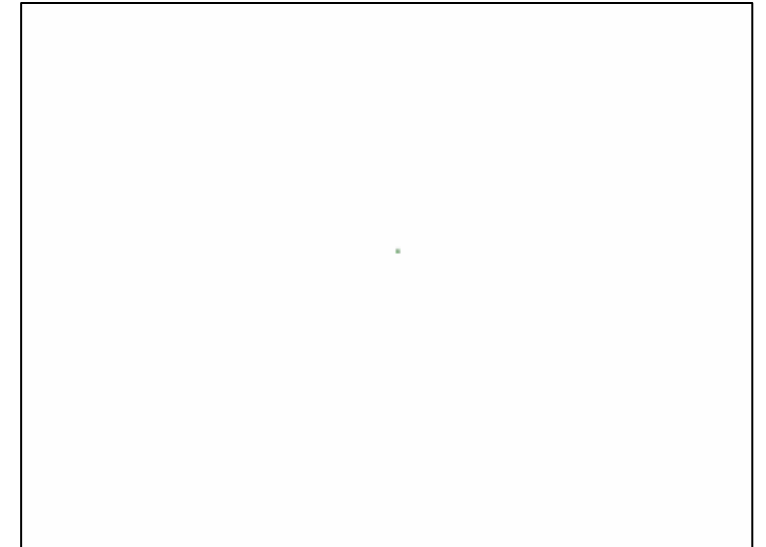
Sampling-based Path Planners

Rapidly-exploring Random Trees (RRT):

- Build a tree incrementally from a start configuration
- Widely used in robotics and motion planning
- Can handle obstacles
- Efficiently explores high-dimensional spaces

Drawbacks:

- Jagged, infeasible paths
- Might be too slow, especially in Python



Sampling-based Path Planners

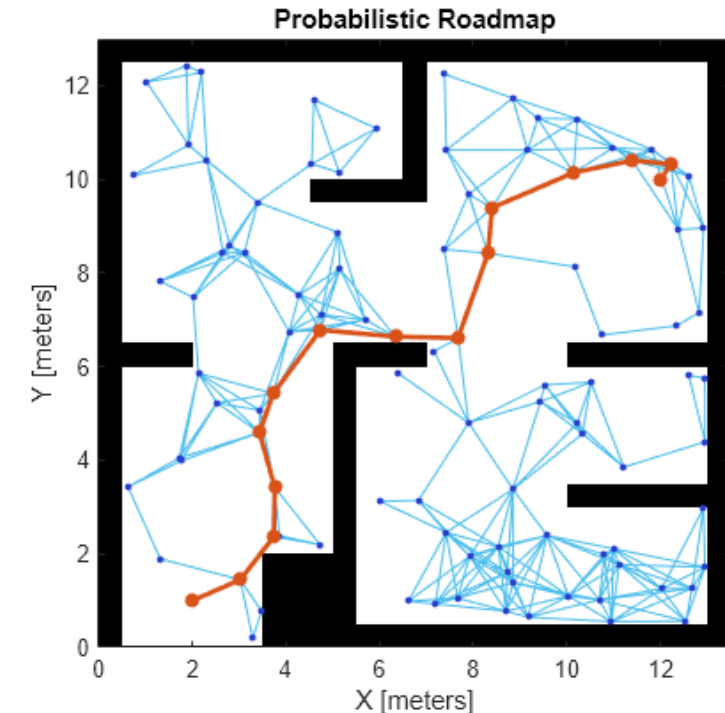


Probabilistic Roadmaps (PRM):

- Mostly used for multi-query path planning
- Two phases:
 - Roadmap construction
 - Path query
- Efficient for static environments with multiple start/goal pairs

Drawbacks:

- Recompute on environment change
- Costly for single queries, has high initial computation cost





Polynomial Trajectory Path Planners

Cubic Splines (already implemented)

- Super easy to connect waypoints
- Smooth derivatives for velocity and acceleration
- But: Suboptimal

Minimum Snap Trajectories

- Idea: Utilize higher order polynomials while minimizing snap (4th derivative of position)
- Easy available as python package [1]
- Section times can be optimized for good performance [2]
- But: Can become infeasible

[1] <https://pypi.org/project/minsnap-trajectories>

[2] <https://ieeexplore.ieee.org/document/5980409>



Trajectory Path Planners

Point Mass Model (PMM) Trajectory Planner

- Approximates drone as point mass with acceleration constraints
- Generates near time-optimal trajectories
- Computationally efficient (compared to full model planners)
- Existing open source C++ implementation [1]
- But: Can become infeasible

Many other approaches

- Some combine trajectory planning with obstacle avoidance
- Good summary and list of open source resources can be found in [2]

[1] https://github.com/ctu-mrs/pmm_uav_planner

[2] <https://arxiv.org/pdf/2301.01755>



Should I Use... Trajectory Planning?

Requirements

- Basics on path planning (Splines, smoothness)
- Some optimization experience

Nice to have

- Experience with obstacle avoidance
- Model-based sampling



What We Cover Today

1. Choosing The Right Method [10 min]
2. Path Planners [15 min]
3. Model Predictive Control [15 min]
4. Sampling-Based Model Predictive Control [10 min]
5. Deep Reinforcement Learning [15 min]
6. Next Steps & Questions [5 min]



What We Cover Today

1. Choosing The Right Method [10 min]
2. Path Planners [15 min]
3. Model Predictive Control [15 min]
4. Sampling-Based Model Predictive Control [10 min]
5. Deep Reinforcement Learning [15 min]
6. Next Steps & Questions [5 min]



Model Predictive Control (MPC)

- Idea: Compute control actions for a dynamic system that minimize a cost function over a given horizon
- MPC solves the constrained optimization over the whole horizon
- Only applies the first action, then repeats in the next step
- Pro:
 - Can handle constrained nonlinear systems
 - Can guarantee constraint satisfaction on inputs and states
- Con:
 - Computationally expensive for nonlinear systems / large horizons



Model Predictive Control (MPC)

- Cost function consists of the sum over the stage cost along the given horizon and the terminal cost in the terminal state
- The cost is minimized given the system dynamics f and under the constraints

$$\begin{aligned} J_k^*(x_k) = & \min_{u_{k|k}, \dots, u_{k+N-1|k}} \underbrace{\ell_N(x_{k+N|k})}_{\text{Terminal cost}} + \underbrace{\sum_{i=0}^{N-1} \ell(x_{k+i|k}, u_{k+i|k})}_{\text{Finite horizon}} \\ \text{subject to } & x_{k+i+1|k} = f(x_{k+i|k}, u_{k+i|k}), \quad \forall i \in \{0, \dots, N-1\}, \\ & x_{k+i|k} \in \mathcal{X}, u_{k+i|k} \in \mathcal{U}, \quad \forall i \in \{0, \dots, N-1\}, \\ & \underbrace{x_{k+N|k} \in \mathcal{X}_f}_{\text{Terminal constraint}}, \\ & x_{k|k} = x_k, \end{aligned}$$



MPC in Practice – Soft Constraints

- State constraints can lead to infeasibility in practice if the states leaves the set of feasible states
- Reasons: Noise, disturbance, model mismatch...
- Solution: Soften state constraints to prevent infeasibility if a constraint is violated, encourage satisfaction by adding a slack cost to the cost function

$$\begin{aligned} J_k^*(x_k) = \min_{u_{k|k}, \dots, u_{k+N-1|k}, \epsilon} & \ell_N(x_{k+N|k}) + \sum_{i=0}^{N-1} \ell(x_{k+i|k}, u_{k+i|k}) + \underbrace{l(\epsilon)}_{\text{Slack cost}} \\ \text{subject to} & \quad x_{k+i+1|k} = Ax_{k+i|k} + Bu_{k+i|k}, \quad \forall i \in \{0, \dots, N-1\}, \\ & \quad Fx_{k+i|k} \leq f_x + \epsilon_i, \quad \forall i \in \{0, \dots, N-1\}, \\ & \quad \epsilon_i \geq 0, \quad \forall i \in \{0, \dots, N-1\}, \quad \text{Soft state constraint} \\ & \quad u_{k+i|k} \in \mathcal{U}, \quad \forall i \in \{0, \dots, N-1\}, \\ & \quad Lx_{k+N|k} \leq l_x + \epsilon_f, \quad \leftarrow \text{Soft terminal constraint} \\ & \quad \epsilon_f \geq 0, \\ & \quad x_{k|k} = x_k, \end{aligned}$$



MPC in Practice – Input Rate Constraints

- Since the input cannot change arbitrarily fast in the real system, we can constrain the change of our control input in the MPC
- We extend the state of the system to include the previous inputs and use the derivative of the previous inputs as new inputs:

$$\begin{aligned} J_k^*(x_k, x_{u,k}) = & \min_{\Delta u_{k|k}, \dots, \Delta u_{k+N-1|k}} \ell_N(x_{k+N|k}) + \sum_{i=0}^{N-1} \ell_{\Delta}(x_{k+i|k}, \Delta u_{k+i|k}) \\ \text{subject to } & x_{k+i+1|k} = f(x_{k+i|k}, u_{k+i|k}), \quad \forall i \in \{0, \dots, N-1\}, \\ & u_{k+i|k} = u_{k+i-1|k} + \Delta u_{k+i|k}, \quad \forall i \in \{0, \dots, N-1\}, \\ & x_{k+i|k} \in \mathcal{X}, \quad \forall i \in \{0, \dots, N\}, \\ & u_{k+i|k} \in \mathcal{U}, \quad \forall i \in \{0, \dots, N\}, \\ & \Delta u_{k+i|k} \in \mathcal{U}_{\Delta}, \quad \forall i \in \{0, \dots, N-1\}, \\ & x_{k|k} = x_k, \\ & u_{k-1|k} = u_{k-1}, \end{aligned}$$

Input dynamics

Input rate constraint



MPC in Practice – Impact of Solvers and Parameters

- Solutions for non-linear optimizations can look vastly different depending on
 - Your solver
 - Solver parameters
 - Cost function
 - Horizon
 - ...
- Make your optimization problem simple at first and check if the MPC produces something reasonable!



Should I Use... MPC?

Requirements

- Solid knowledge of model-based control
- Fundamentals of constrained optimization
- Good python skills

Nice to have

- CasADi and acados experience



MPC Tools

CasADi [1]

- Symbolic framework for numeric optimization
- Can be used for optimal control with Python bindings
- Powerful solvers (such as IPOPT) but often not optimized to work in real time

Acados [2]

- Optimal Control Framework, but optimized for real-time applications
- Built on symbolic CasADi models
- Note: Documentation suboptimal → Example provided

Drone Models and Controllers [3,4]

- Implementation in the ArrayAPI standard (JAX, PyTorch) and symbolically (CasADi)
- Low sim2real gap for the used drone (Crazyflie brushless)

[1] <https://web.casadi.org/>

[2] <https://github.com/acados/acados>

[3] <https://github.com/learnsyslab/drone-models>

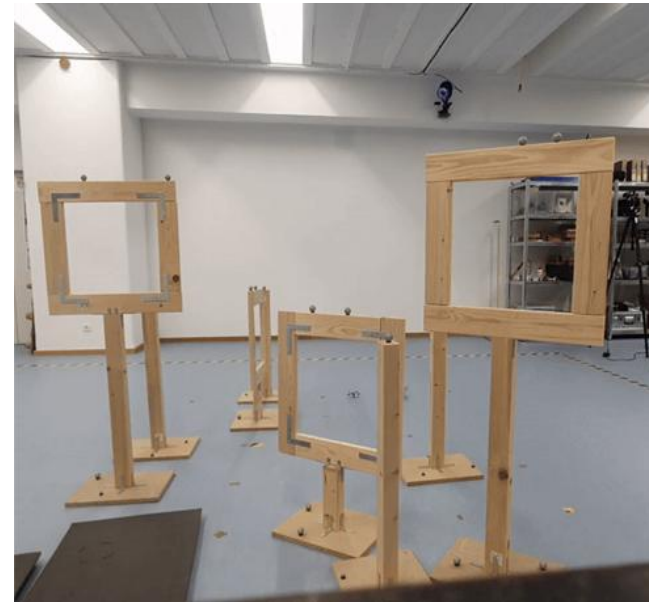
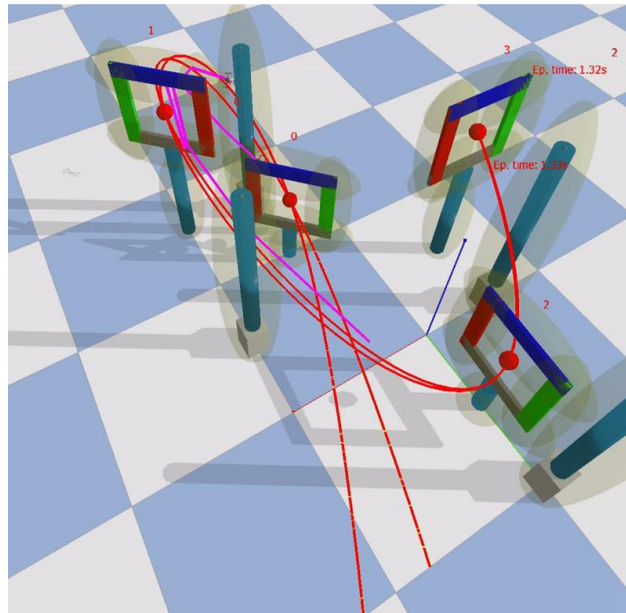
[4] <https://github.com/learnsyslab/drone-controllers>



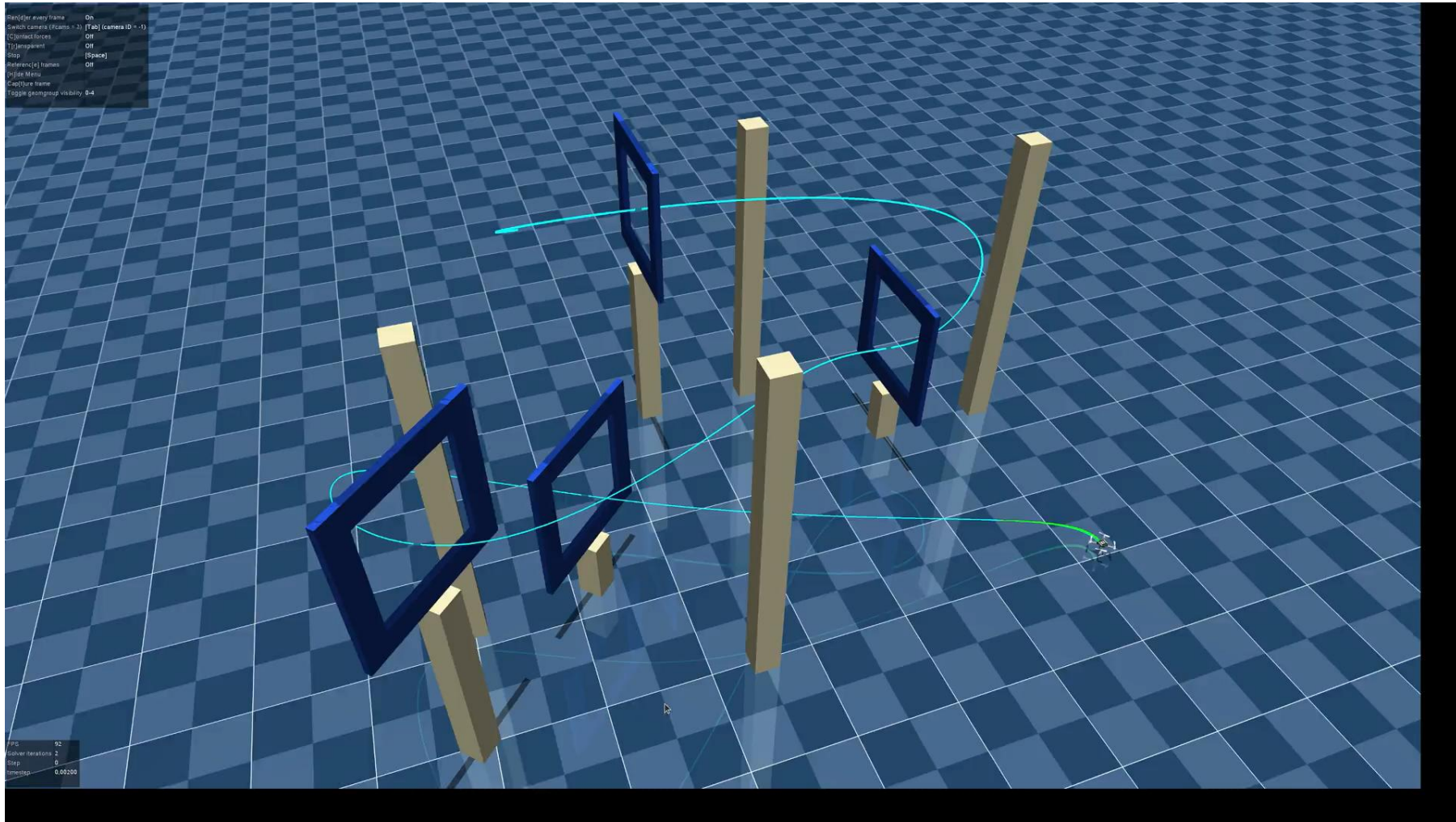
MPC – One last Tip

Visualize! We have an awesome simulation – you can use it to draw trajectories as well as MPC horizons & other things:

- Helps you debug
- Looks great in your intermediary/ final presentations



MPC – One last Tip





MPC Drone Racing Papers

- Initial works: E. Kaufmann, M. Gehrig, P. Foehn, R. Ranftl, A. Dosovitskiy, V. Koltun, and D. Scaramuzza, "[Beauty and the beast: Optimal methods meet learning for drone racing](#)," 2019 International Conference on Robotics and Automation (ICRA)
- MPCC (improve trajectory on the fly): A. Romero, S. Sun, P. Foehn, and D. Scaramuzza, "[Model predictive contouring control for time-optimal quadrotor flight](#)," IEEE Transactions on Robotics, 2022.
- Tunnel approaches: J. Arrizabalaga and M. Ryll, "[Towards Time-Optimal Tunnel-Following for Quadrotors](#)," 2022 International Conference on Robotics and Automation (ICRA).
- MPCC++ (current model-based SotA): M. Krinner, A. Romero, L. Bauersfeld, M. Zeilinger, A. Carron, D. Scaramuzza, "[MPCC++: Model Predictive Contouring Control for Time-Optimal Flight with Safety Constraints](#)", Robotics: Science and Systems (RSS) 2024.



What We Cover Today

1. Choosing The Right Method [10 min]
2. Path Planners [15 min]
3. Model Predictive Control [15 min]
4. Sampling-Based Model Predictive Control [10 min]
5. Deep Reinforcement Learning [15 min]
6. Next Steps & Questions [5 min]



What We Cover Today

1. Choosing The Right Method [10 min]
2. Path Planners [15 min]
3. Model Predictive Control [15 min]
4. Sampling-Based Model Predictive Control [10 min]
5. Deep Reinforcement Learning [15 min]
6. Next Steps & Questions [5 min]



Sampling-Based MPC

- Problem with regular MPC: Solvers need convex costs to converge properly
- Idea: Just try out (i.e., sample) many different possible control sequences
- The best sample (by cost) is used for the next control input
- Pros:
 - Can handle nonconvex and sparse costs
 - Can handle cluttered environments
 - Control compute time is almost constant
- Cons:
 - Many useless samples (lots of samples necessary)
 - Might not find *the* optimal solution



Cross-Entropy Method (CEM) SMPC

- Instead of picking the single best sample
⇒ fit Gaussian distribution to the best K samples (μ_t, Σ_t)
- Apply mean input (μ_t)
- In the next control step
 - Shift the sequence by one step ($\mu_t \leftarrow \mu_{t-1}$) and add a constant ($\Sigma_t \leftarrow \Sigma_{t-1} + \Sigma_{\text{init}}$)
(The last value stays constant)
 - Sample new sequence from distribution
 - Restart
 - Note: Some CEM implementations do multiple sampling/fitting iterations per control step

⇒ Instead of blindly sampling each time, we reuse prior information [1]



Model Predictive Path Integral (MPPI)

- CEM SMPC treats all top K samples equal
⇒ What if the optimal K is larger or smaller?
- Exponential weighted sampling:
$$\mu_t = \frac{\sum_i w_i s_i}{\sum_i w_i}, \text{ where } w_i = \exp\left(-\frac{s_i}{\lambda}\right) \text{ with the temperature } \lambda$$

⇒ We don't throw away samples as in CEM SMPC [1]
- While in the classic formulation, the covariance Σ is constant, modern methods lean to online adaption [2]



- Smooth input samples
 - Low-pass filtering of sampled inputs [1]
 - Temporally-correlated noise [2]
- Increasing sample efficiency
 - Reusing of previous elites [2]
 - Deterministic sampling [3]

[1] Piotr Kicki, "[Low-pass sampling in Model Predictive Path Integral Control](#)", 2026 International Conference on Robotics and Automation (ICRA)

[2] C. Pinneri, S. Sawant, S. Blaes, J. Achterhold, J. Stueckler, M. Rolinek, G. Martius, "[Sample-efficient Cross-Entropy Method for Real-time Planning](#)", 2021 Conference on Robot Learning (CoRL)

[3] M. Walker, D. Frisch, U. D. Hanebeck, "[Sample-Efficient and Smooth Cross-Entropy Method Model Predictive Control Using Deterministic Samples](#)", 2025, arXiv



Should I Use... SMPC?

Requirements

- Solid knowledge of model-based control
- Solid knowledge of probability theory
- Knowledge in JAX or PyTorch
- Good python skills



PyTorch MPPI [1]

- MPPI implementation in PyTorch
- Uses approximate models

JAX MPPI [2]

- Two implementations in JAX: CEM and MPPI
- Highly efficient

Drone Models and Controllers [3,4]

- Implementation in the ArrayAPI standard (JAX, PyTorch) and symbolically (CasADi)
- Low sim2real gap for the used drone (Crazyflie brushless)

[1] https://github.com/UM-ARM-Lab/pytorch_mppi

[2] <https://github.com/jlehtomaa/jax-mppi>

[3] <https://github.com/learnsyslab/drone-models>

[4] <https://github.com/learnsyslab/drone-controllers>



- Racing: G. Williams, B. Goldfain, P. Drews, J. M. Rehg, E. A. Theodorou, “[Autonomous Racing with AutoRally Vehicles and Differential Games](#)”, arXiv, 2017
- Racing: J. Yin, Z. Zhang, E. Theodorou, P. Tsiotras, “[Trajectory Distribution Control for Model Predictive Path Integral Control using Covariance Steering](#)”, 2022 International Conference on Robotics and Automation (ICRA)
- MPPI on quadrotors: Y. Zhai, R. Reiter, and D. Scaramuzza, “[PA-MPPI: Perception-Aware Model Predictive Path Integral Control for Quadrotor Navigation in Unknown Environments](#)”, IEEE Robotics and Automation Letters (RA-L), 2026



What We Cover Today

1. Choosing The Right Method [10 min]
2. Path Planners [15 min]
3. Model Predictive Control [15 min]
4. Sampling-Based Model Predictive Control [10 min]
5. Deep Reinforcement Learning [15 min]
6. Next Steps & Questions [5 min]



What We Cover Today

1. Choosing The Right Method [10 min]
2. Path Planners [15 min]
3. Model Predictive Control [15 min]
4. Sampling-Based Model Predictive Control [10 min]
5. Deep Reinforcement Learning [15 min]
6. Next Steps & Questions [5 min]



Deep Reinforcement Learning (RL)

- Objective: Maximize expected, cumulative (often discounted) future reward
- Tabular methods cannot deal with the required dimensionality
→ Use neural networks as approximators
- For continuous actions: Policy gradient methods
- PPO, SAC, TD3

$$\mathbb{E}_{\pi_{\theta}} \left[\sum_{k=1}^T r(x_k, u_k) | x_1 = x \right]$$



Deep RL – Pros

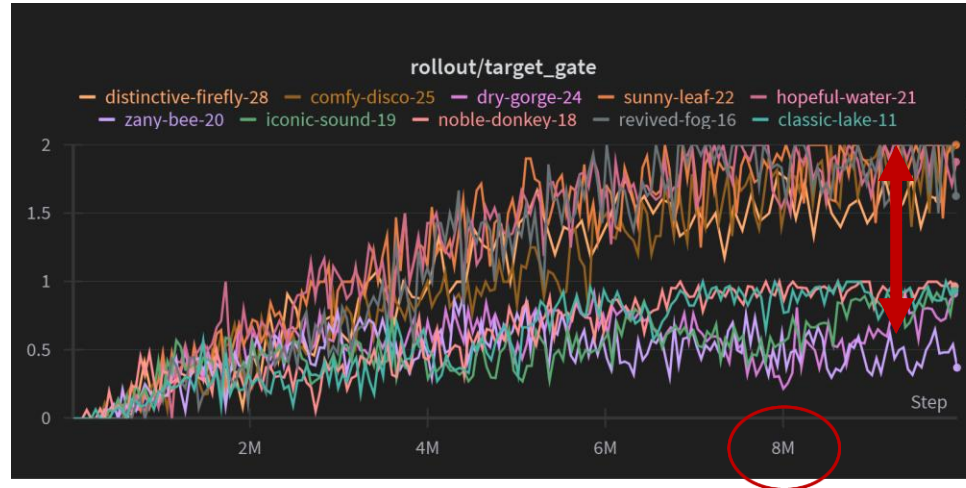
- No model required, only a simulation and a reward function
- Policy is a simple neural network → Low inference compute
- No prior trajectory required
- Can be trained for robustness against model mismatches



Deep RL – Cons

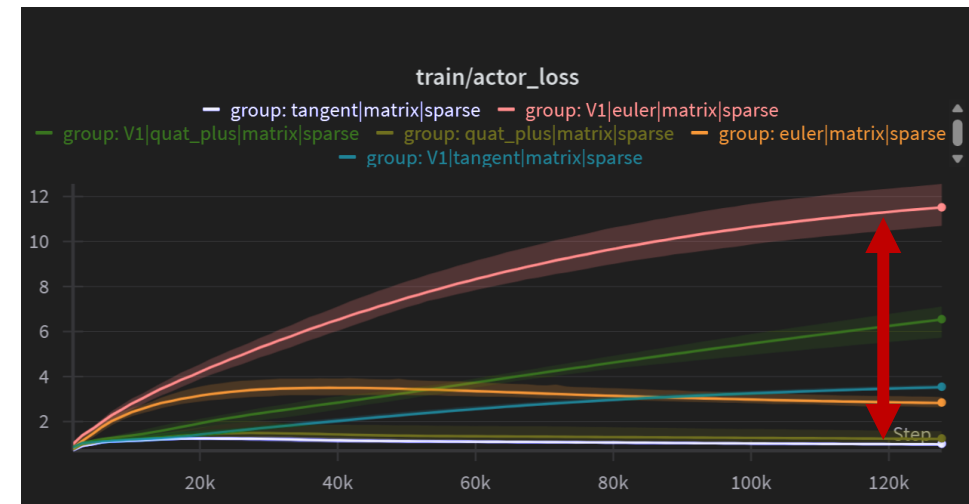
- Training a good RL policy is *hard*
- Sample requirement is extremely high
- Training is often unstable – the same settings with different seeds might not converge!
- Training dynamics are unintuitive
- Small changes in the reward function can produce vastly different outcomes

Deep RL – Cons



Sometimes large spreads due to seed differences

Training might take a lot of steps



Action space plays a role [1]

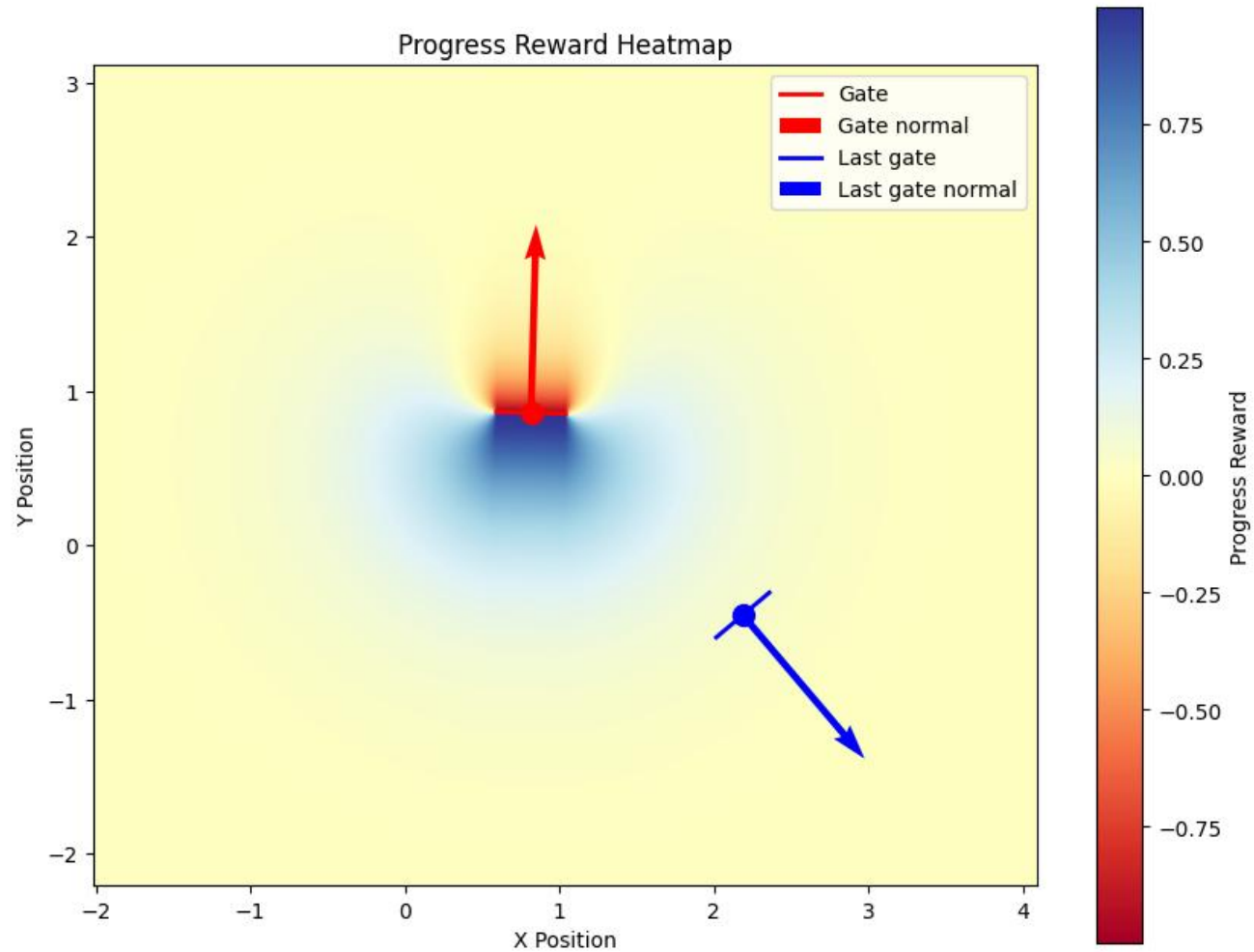
[1] <https://arxiv.org/abs/2510.11103>



Deep RL – Making it Work

- You may need larger batch sizes than you think (10k+ is not unusual)
- Have sufficient compute!
- Use existing implementations. Coding from scratch is extremely error-prone
- Think about your assumptions and reward:
 - Markov property of the observation
 - Markov property of the reward
 - "Will to live" and discount factors
 - Dense vs sparse rewards

Deep RL – Making it Work





Deep RL – Making it Work

- Observations for Neural Networks:
 - Do you have to normalize? What do you normalize?
 - Transform non-smooth representations [1]
 - One-hot encode categorical variables
 - ...
- Think about your actions:
 - Use a sensible action space, i.e., try to avoid roll-pitch-yaw angles [2]
 - Be careful about the amount of randomization
 - Limit to a useful range
 - Design your policy so that it can only output valid values

[1] <https://arxiv.org/pdf/2409.11935>

[2] <https://arxiv.org/abs/2510.11103>



Deep RL – Making it Work

- Wrap your environment to modify actions and observations for training (see gymnasium wrappers [1])
- Log hyperparameters (e.g., with wandb [2])
- Start with a simple task (e.g., hover only [3])
- Use default learning parameters at the beginning, then modify
- The reward function may require several iterations. Pay attention to scales!
- For hardware deployment: Domain randomization!

[1] <https://gymnasium.farama.org/api/wrappers/>

[2] <https://wandb.ai/>

[3] <https://arxiv.org/abs/2311.13081>



Should I Use... Deep RL?

Requirements

- Deep learning experience in PyTorch or JAX
- Already solved an environment (Cartpole, Lunar Lander, ...) with Q-learning or policy gradient methods
- Knowledge of RL theory (MDPs, Bellman equation, value function approximation)

Big plus

- Experience with implementing the algorithms yourself



Deep RL Tools

- [Gymnasium](#): Standard API for environments and wrappers
- [Stable-baselines3](#): High-quality RL implementations, but hard to modify
- [CleanRL](#): Single-file implementations of algorithms. Very good starting point
- [Spinning-up](#): OpenAI learning resources for deep RL. Recommended to understand algorithms without reading the full papers
- [Wandb](#): Live ML configs and metrics logging + hyperparameter optimization



Deep RL Drone Racing Papers

- Standard RL approach: Y. Song, M. Steinweg, E. Kaufmann, and D. Scaramuzza, [“Autonomous Drone Racing with Deep Reinforcement Learning,”](#) IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2021.
- Follow-up: E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, [“Champion-level drone racing using deep reinforcement learning,”](#) Nature, vol. 620, 2023.
- Comparison to MPC: Y. Song, A. Romero, M. Mueller, V. Koltun, and D. Scaramuzza, [“Reaching the limit in autonomous racing: Optimal control versus reinforcement learning,”](#) Science Robotics, 2023.
- Combination with MPC: A. Romero, Y. Song, and D. Scaramuzza, [“Actor-critic model predictive control,”](#) IEEE International Conference on Robotics and Automation (ICRA), 2024.



What We Cover Today

1. Choosing The Right Method [10 min]
2. Path Planners [15 min]
3. Model Predictive Control [15 min]
4. Sampling-Based Model Predictive Control [10 min]
5. Deep Reinforcement Learning [15 min]
6. Next Steps & Questions [5 min]



What We Cover Today

1. Choosing The Right Method [10 min]
2. Path Planners [15 min]
3. Model Predictive Control [15 min]
4. Sampling-Based Model Predictive Control [10 min]
5. Deep Reinforcement Learning [15 min]
6. Next Steps & Questions [5 min]



Next Steps

- Lecture on coding and intro to our hardware setup
- Videos on specific topics like report writing
- Lab sessions
 - We will have a sign-up form every week in Moodle
 - Slots will be announced at least a day in advance
 - Students must attend the hardware setup lecture

Timeline



Week	Dates	Content	Deadlines	Notes	Other Dates
1	15/04/2026	Lec: Kickoff		w/ Angela	
2	22/04/2026	Lec: Project intro and setup		w/ Marcel	
3	29/04/2026	<i>Office Hour</i>	29/04/2026 Submit to online challenge	w/ Tutors	
4	06/05/2026	Lec: Selected algorithms for racing		w/ Marcel	
5	13/05/2026	Lec: Hardware, code, tools + Lab tour		w/ Martin	
6	20/05/2026	<i>Lab sessions</i>			
7	27/05/2026	<i>Lab sessions</i>			
8	03/06/2026	<i>Lab sessions</i>			ICRA 01.06-05.06
9	10/06/2026	Progress presentations	09/06/2026 Submit slides		
10	17/06/2026	<i>Lab sessions</i>			L4DC 17.06 - 19.06
11	24/06/2026	<i>Lab sessions</i>			
12	01/07/2026	<i>Lab sessions</i>			Demo Day 02.07.
13	08/07/2026	Final Presentations	07/07/2026 Submit report, slides, code		

Questions?